



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space is the complete set of all possible states that the agent can reach in the environment.

The search tree is the tree generated by the search algorithm while trying to find a path from the starting state to the goal state. The same state can appear more than once in a search tree through different paths. The reached set in my code helps avoid visiting the same state repeatedly.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set is used to keep track of states that have already been visited.

This changes the algorithm from a tree search into a graph search and prevents the agent from repeatedly exploring the same states.

Without the reached set, DFS could keep moving between the same positions in a cycle. This could cause it to explore the same states repeatedly and potentially continue indefinitely without finding the goal.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS generally produces a more direct and shortest path, while DFS can produce a longer and more winding path.

BFS uses a FIFO queue, so it explores all nodes at one depth before moving to the next depth. Since every movement in my grid has a cost of 1, the first time BFS reaches the food, it has found the path with the minimum number of movements.

DFS uses a LIFO stack, so it follows one branch as deeply as possible before going back and trying another branch. Because of this, it can find the food using a longer path even when a shorter path exists.

Therefore, BFS is optimal for this grid because all actions have the same cost, while DFS is not guaranteed to find the shortest path.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS has a much larger memory requirement because it stores many nodes in its frontier at the same time. Its space complexity is approximately $O(b^d)$, where b is the branching factor and d is the depth of the shallowest solution.

DFS uses a stack and mainly stores the current path and other unexplored branches. Its space complexity is approximately $O(bm)$, where m is the maximum depth of the search.

Therefore, on a massive 1000×1000 grid, BFS is more likely to run out of memory because it may have to store a very large number of states at once. DFS requires much less memory, although it can take longer and may explore a poor path before finding the food.

In the implementation, both algorithms also use a reached set, so memory is additionally required to store the states that have already been explored.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING